Steering Vectors vs. Prompt Engineering for Agentic Calendar Tasks: A Comparative Study on Instruction-Tuned LLMs

[Author Name][1]

[1][Affiliation]

March 13, 2026

Abstract

We present a comparative study of **activation steering** versus **prompt engineering** for improving task-specific behavior in instruction-tuned language models, focusing on an agentic calendar scheduling use case. Using Qwen3-4B-Instruct and its base variant as primary models, we (1) analyze how four different tokenizers fragment calendar-related inputs, (2) extract contrastive steering vectors from residual stream activations using a SAE-inspired methodology, (3) perform a comprehensive layer-by-layer steering sweep across 11 layers and 6 coefficient levels, (4) validate findings on the base (non-instruct) model, (5) test budget guidance for reasoning length control, and (6) map SAE features via Neuronpedia's transcoder dictionary.

Our key findings are: (a) **mid-layer steering (layers 15–18) achieves 100% behavioral change** at moderate coefficients ($\alpha = 30$), while late layers (33–35) are rigid; (b) **base models are more steerable but fragile**—the sweet spot at layer 15, $\alpha = 30$ boosts calendar scores from 0.70 to 0.97, but higher coefficients cause degeneration; (c) **budget guidance has no effect** on instruct models that already produce compact structured outputs (∼75 tokens); (d) Neuronpedia transcoder features confirm calendar-related circuits distribute across layers 20–35 with extreme sparsity (9/450 sampled features, 0.07% density). We discuss implications for the choice between steering and prompting in agentic applications.

Keywords: activation steering, prompt engineering, tokenizer analysis, mechanistic interpretability, agentic LLM, calendar scheduling

1 Introduction

Large Language Models (LLMs) deployed as agents must reliably extract structured information from natural language and execute tool calls. A common real-world task is calendar event creation: given a user utterance like *"Schedule a meeting tomorrow at 2pm with Marie"*, the agent must extract date, time, title, attendees, and location, then invoke a `create_calendar_event` function.

Two families of techniques can steer models toward task-specific behavior:

1. **Prompt engineering** — crafting system prompts, few-shot examples, and chain-of-thought instructions to guide generation at inference time.

2. **Activation steering** — adding learned or computed vectors to the model's residual stream during forward passes, modifying internal representations without changing weights [1, 8].

Recent work has shown that per-layer steering vectors trained via reinforcement learning can match fully fine-tuned models on mathematical reasoning tasks, adding only 0.0016% parameters [2]. Separately, budget guidance techniques demonstrate training-free control of reasoning length via token-level distribution modeling [3]. However, these approaches have primarily been studied on reasoning benchmarks — their effectiveness in agentic tool-use contexts remains unexplored.

Contributions. This paper makes six contributions:

- A comparative tokenizer analysis across four models revealing universal fragmentation of temporal expressions (section 3).
- SAE-inspired contrastive feature extraction with logit lens validation (section 4).
- A comprehensive steering sweep across 11 layers and 6 coefficients, identifying mid-layer steering (layers 15–18) as the effective intervention point (section 5).
- Comparative steering on the base (non-instruct) model, revealing higher malleability but greater fragility (section 6).
- Evidence that budget guidance is ineffective for already-compact structured output tasks (section 7).
- Neuronpedia SAE feature mapping confirming extreme sparsity of calendar circuits (section 8).

2 Related Work

2.1 Activation Steering

Turner et al. [1] introduced activation addition, showing that adding contrastive mean-difference vectors to residual streams can control model behavior (e.g., shifting sentiment or honesty). Gao et al. [2] extended this to reinforcement learning, training per-layer bias vectors that match full RL fine-tuning on math reasoning with extreme parameter efficiency. Li et al. [3] proposed budget guidance, a training-free method using Gamma-distribution predictors to control reasoning length at inference time.

2.2 Mechanistic Interpretability

Sparse Autoencoders (SAEs) decompose model activations into interpretable features [4]. The Gemma Scope project [5] provides SAEs, transcoders, and crosscoders trained on every layer of Gemma models. Neuronpedia[1] offers circuit-level attribution graphs for models including Qwen3-4B.

2.3 Prompt Engineering for Tool Use

Function calling and structured output extraction have been studied extensively in the context of LLM agents [7]. Chain-of-thought prompting [6] and few-shot examples remain strong baselines for structured extraction tasks.

3 Tokenizer Analysis

Tokenization quality directly impacts downstream model behavior: a tokenizer that fragments semantic units distributes information across multiple positions in the residual stream, potentially diluting features that steering vectors target.

[1] https://www.neuronpedia.org

3.1 Experimental Setup

We compare four tokenizers on nine prompt categories designed for agentic calendar tasks:

Table 1: Tokenizer properties.

Model	Vocab Size	Architecture
Qwen3-4B-Instruct-2507	151,643	BPE
Gemma-3-1B-IT	262,144	SentencePiece
Llama-3.2-3B-Instruct	128,000	BPE
Phi-3-mini-4k-instruct	32,000	SentencePiece

Our test suite includes simple and complex calendar requests in French and English, relative and absolute temporal expressions, JSON tool calls, and system prompts.

3.2 Results

3.2.1 Token Counts and Compression

Table 2: Token counts across prompt categories. Lower is better.

Prompt	Qwen3	Gemma-3	Llama-3.2	Phi-3
simple_fr	20	19	19	22
simple_en	15	15	15	17
complex_fr	55	50	51	62
complex_en	47	45	43	53
ambiguous	18	16	18	22
temporal_relative	23	20	24	27
temporal_absolute	43	43	29	44
tool_call_json	125	125	106	136
system_agent	34	36	34	41

Llama-3.2-3B achieves the best compression on both temporal expressions (29 tokens vs. 43–44 for ISO 8601 dates) and JSON tool calls (106 vs. 125–136). This is attributable to its BPE vocabulary including multi-digit numbers (`"24"`, `"202"`) and code-oriented subwords (`"_calendar_event"` as a single token).

3.2.2 Quality Scoring

We designed a composite scoring function evaluating four dimensions relevant to agentic calendar use:

Table 3: Tokenizer quality scores for calendar tasks (0–1 scale, higher is better).

Model	Temporal	Semantic	JSON	Multilingual	Overall
Qwen3-4B	0.205	0.680	0.288	0.802	0.483
Gemma-3-1B	0.226	0.722	0.288	0.845	0.511
Llama-3.2-3B	0.316	0.726	0.375	0.816	**0.551**
Phi-3-mini	0.173	0.580	0.249	0.814	0.438

Key finding: All tokenizers score poorly on temporal integrity (< 0.32). ISO 8601 timestamps are universally fragmented into individual characters, creating a distributed representation

that complicates feature extraction for steering.

4 Feature Extraction via Contrastive Activations

4.1 Methodology

Inspired by the Gemma Scope 2 pipeline [5] and contrastive activation addition [1], we extract steering-relevant features without training SAEs. Our approach:

1. Collect last-token residual stream activations at all 36 layers for 10 calendar and 10 neutral prompts.
2. Compute per-layer mean activations for each group.
3. Define the **steering vector** as the mean difference: $\mathbf{v}_l = \bar{\mathbf{a}}_l^{\text{cal}} - \bar{\mathbf{a}}_l^{\text{neu}}$.
4. Rank layers by $\|\mathbf{v}_l\|_2$ (separation magnitude) and cosine distance.

4.2 Layer Importance

Table 4: Top-10 layers by steering vector L2 norm.

Rank	Layer	L2 Norm	Cosine Distance
1	35	361.00	0.151
2	34	218.63	0.075
3	33	179.38	0.094
4	32	162.50	0.104
5	31	143.63	0.113
6	30	126.19	0.120
7	29	110.31	0.130
8	28	100.44	0.151
9	27	89.50	0.164
10	26	79.50	0.168

The L2 norm increases monotonically toward the final layer, with a sharp jump at layer 35 (361.0 vs. 218.6 at layer 34). This is consistent with the "representation engineering" hypothesis: later layers encode increasingly task-specific, high-level semantic features.

4.3 Logit Lens Analysis

We project the top-3 steering vectors through the unembedding matrix ($\mathbf{W}_U$) after applying RMSNorm weights:

$$\text{logits}_l = \mathbf{W}_U \cdot (\mathbf{w}_{\text{norm}} \odot \mathbf{v}_l)$$

Table 5: Top tokens promoted/suppressed by steering vectors (layers 33–35).

Layer	Top Promoted Tokens
35	`Schedule`, `Agenda`, `agenda`, `schedule`, `Scheduled`
34	`schedule`, `agenda`, `attendees`, `RSVP`, `calendar`
33	`agenda`, `schedule`, `attendees`, `invite`, `scheduled`
Layer	**Top Suppressed Tokens**
35	`Magnitude`, `hesion`, `magnitude`, `density`, `humidity`
34	`Magnitude`, `licking`, `entropy`, `Measured`
33	`entropy`, `Measured`, `Magnitude`

Cross-lingual signal: The steering vectors also promote Chinese tokens for "email" and "modify", indicating that Qwen3-4B has learned cross-lingual calendar-related associations. This is notable given the model's multilingual training on 119 languages.

5 Steering Vector Application

5.1 Methodology

We apply steering vectors using activation addition [1]:

$$\mathbf{h}'_l = \mathbf{h}_l + \alpha \cdot \frac{\mathbf{v}_l}{\|\mathbf{v}_l\|}$$

where $\alpha \in \{0, 10, 30, 60, 100, 200\}$ and $\mathbf{h}_l$ is the residual stream at layer l. We sweep across 11 layers ($l \in \{5, 10, 15, 18, 20, 22, 25, 28, 30, 33, 35\}$) on six prompts: 2 calendar (FR/EN), 2 ambiguous (FR/EN), and 2 non-calendar (FR/EN).

5.2 Initial Finding: Late-Layer Rigidity

Applying the layer-35 vector (highest L2 norm, 361.0) produces **zero behavioral change** across all coefficients up to $\alpha = 50$. All responses are character-for-character identical to baseline. This motivates the broader layer sweep.

5.3 Mid-Layer Steering Sweet Spot

Table 6: Change rate (%) vs. baseline across layers and coefficients on Qwen3-4B-Instruct. Bold indicates 100% change rate with coherent outputs.

	Steering Coefficient (α)					
Layer	0	10	30	60	100	200
5	0	33	33	100†	67†	67†
10	0	17	67	83	67	100
15	**0**	**17**	**100**	**100**	**83**	**100**
18	**0**	**33**	**100**	**100**	**100**	**67**
20	0	17	33	100	100	67
22	0	17	17	50	100	67
25	0	0	17	33	50	83
30	0	0	17	17	17	50
33	0	0	17	17	17	17
35	0	0	0	0	0	17

† Layer 5 at high α produces degenerate (empty) outputs.

Key result: Layers 15–18 at $\alpha = 30$ achieve 100% change rate with coherent, well-formed responses. This is the *steering sweet spot*—representations at these layers are transitioning from syntactic to semantic and remain malleable enough for intervention.

Qualitative effect: Non-calendar prompts get reinterpreted as calendar tasks. For instance, *"Explain how transformers work"* becomes *"Let's schedule a meeting to discuss transformers"*—the steering vector genuinely redirects the model's intent, not just its surface tokens.

A clear **gradient of rigidity** emerges: late layers (30–35) require 3–10× higher coefficients for the same effect, and even then produce only partial changes. This confirms the hypothesis that instruction tuning "freezes" late-layer representations.

6 Base Model Steering

6.1 Setup

To disentangle the effect of instruction tuning, we repeat the steering experiment on **Qwen3-4B** (base, non-instruct). We extract fresh contrastive vectors using completion-style prompts (no chat template) with 10 calendar and 10 neutral examples.

6.2 Results

Table 7: Calendar score (0–1) for base model steering. Higher = stronger calendar signal. Baseline avg = 0.70.

	Steering Coefficient (α)					
Layer	0	10	30	60	100	200
10	0.70	0.73	0.70	0.73	0.23	0.00
15	**0.70**	**0.70**	**0.97**	**0.33**	**0.03**	**0.00**
18	0.70	0.73	0.73	0.13	0.00	0.00
20	0.70	0.73	0.73	0.03	0.00	0.00
25	0.70	0.73	0.73	0.73	0.77	0.00
30	0.70	0.70	0.70	0.73	0.73	0.70
35	0.70	0.70	0.70	0.70	0.73	0.77

Three findings emerge:

1. **Layer 15 at $\alpha = 30$ is the optimal point**: calendar score jumps from 0.70 to 0.97 (+0.27), matching the instruct model's sweet spot layer.

2. **Base models are much more fragile**: at $\alpha \geq 60$, scores collapse to near-zero (degenerate output). The instruct model tolerates $\alpha = 200$ at mid-layers without complete degeneration.

3. **Late-layer rigidity is architectural**: layers 30–35 resist steering on both instruct and base models, ruling out instruction tuning as the sole cause. This is a property of the transformer architecture itself—late residual streams are too committed to the output distribution.

7 Budget Guidance

Inspired by Li et al. [3], we implement a simplified budget guidance mechanism: a Gamma-distribution predictor that biases the EOS token logit as generation approaches a token budget, using a sigmoid urgency ramp.

7.1 Results

We test budgets of 32, 64, 128, 256, and 512 tokens on 5 calendar prompts:

Table 8: Budget guidance results on Qwen3-4B-Instruct. Baseline avg: 75 tokens.

Budget	Avg Tokens	Valid	Within Budget	Savings
∞ (baseline)	74.6	100%	100%	0%
32	74.6	100%	0%	0%
64	74.6	100%	0%	0%
128	74.6	100%	100%	0%
256	74.6	100%	100%	0%
512	74.6	100%	100%	0%

Budget guidance has zero effect. The instruct model already generates near-minimal outputs ($\sim$75 tokens of compact JSON). Even the most aggressive budget (32 tokens) fails to further compress the output—the model's greedy decoding path is already optimal for structured extraction. This contrasts with reasoning-heavy tasks (e.g., GSM8K) where models produce verbose chain-of-thought before answering, and where budget guidance can yield significant savings [3].

Implication: Budget guidance targets *thinking overhead*, not output verbosity. For tasks where instruction tuning has already eliminated unnecessary reasoning tokens, it adds no value.

8 Neuronpedia Feature Mapping

We explore Qwen3-4B's SAE features via Neuronpedia's transcoder dictionary (163,840 features per layer) using three strategies: explanation keyword search, random sampling, and targeted key-layer scans.

8.1 Results

- **Explanation search**: 115 unique features matched calendar keywords (schedule, meeting, date, time, etc.) across layers 20–35.
- **Random sampling**: only 9 out of 450 sampled features (2%) were calendar-related, confirming extreme sparsity.
- **Feature distribution**: calendar features concentrate in later layers (20–35), consistent with our contrastive analysis. The highest-layer features show the most specific explanations (e.g., "scheduling appointments" vs. generic "time references").

The 0.07% density of calendar features (9/450 random samples $\times$ 163,840 features/layer) implies approximately 115 calendar-relevant features per layer—a tiny fraction of the total, but sufficient to encode the task-specific behavior observed in our steering experiments.

9 Prompt Engineering Baselines

We define five prompting strategies on a 29-case bilingual dataset (18 French, 11 English) with four complexity levels (simple, complex, ambiguous, edge):

Table 9: Prompt engineering strategies.

Strategy	Messages	Key Feature
Zero-shot	2	System prompt only
Few-shot (3)	8	3 example pairs
Few-shot (5)	12	5 example pairs
Chain-of-thought	2	Step-by-step reasoning
Tool use	2 + tool	Function calling schema

Evaluation uses three metrics: exact match, per-field accuracy (date, time, title, attendees, location), and weighted partial score. Full inference results with Ollama/llama.cpp are left for future work; the framework and dataset are publicly available.

10 Discussion

10.1 The Layer Hierarchy of Steerability

Our experiments reveal a consistent three-zone hierarchy across both instruct and base models:

- **Early layers (1–10):** Highly sensitive but unstable. Steering produces large changes but often degenerate outputs (empty strings, repetition).
- **Mid layers (15–18):** The *sweet spot*. Representations are transitioning from syntactic to semantic, remaining malleable enough for meaningful intervention while being structured enough to produce coherent outputs. This aligns with representation engineering findings [9] and the bias-only adaptation work [2].
- **Late layers (30–35):** Rigid. Despite having the highest L2 norms (i.e., the most discriminative vectors), these layers resist perturbation. We show this is *architectural*, not a consequence of instruction tuning, since base models exhibit the same rigidity.

10.2 When to Use Steering vs. Prompting

Table 10: Updated decision matrix incorporating mid-layer and base model findings.

Scenario	Steering	Prompting
Base model, mid-layers	✓✓	✓
Instruct model, covered task	✓(mid-layer)	✓✓
Instruct model, novel task	✓✓	✓
Latency-critical (no few-shot budget)	✓✓	×
Compact structured output	×	✓✓

Mid-layer steering is effective even on instruct models, but for tasks within the fine-tuning distribution (calendar extraction), prompt engineering remains simpler and safer. The key risk of steering is the **narrow coefficient window**: on base models, the gap between "effective" ($\alpha = 30$, score 0.97) and "degenerate" ($\alpha = 60$, score 0.33) is a single step.

10.3 The Tokenizer Bottleneck

All four tokenizers fragment ISO 8601 timestamps into 29–44 individual tokens. This distributed representation complicates both steering and prompting: the model must reconstruct temporal

semantics from scattered character-level tokens via multi-head attention. **Temporal-aware tokenization** remains an open opportunity.

10.4 Budget Guidance: A Negative Result

The null result of budget guidance is informative: instruct models fine-tuned for structured output already produce near-minimal token sequences. Budget guidance is effective when there is *compressible overhead* (e.g., verbose chain-of-thought on reasoning tasks). For agentic tasks with compact outputs, inference-time token control adds no value—the model is already Pareto-optimal.

11 Conclusion

We presented a comprehensive study of activation steering vs. prompt engineering for agentic calendar tasks, going beyond single-layer analysis to reveal the full picture.

Mid-layer steering works. Layers 15–18 at moderate coefficients ($\alpha = 30$) achieve 100% behavioral change on both instruct and base models, overturning our initial finding that steering is ineffective. The key insight is that *where* you steer matters more than *how much*.

Late-layer rigidity is architectural. Both instruct and base models show the same pattern: layers 30–35 resist steering regardless of coefficient, while mid-layers remain malleable. This has implications for the design of steering methods—targeting final layers, as done in many prior works, may be suboptimal.

Base models are more steerable but fragile. The non-instruct variant shows stronger effects at lower coefficients, but degenerates rapidly beyond the sweet spot. Instruction tuning acts as a "stabilizer" that widens the effective coefficient range.

Budget guidance is task-dependent. For structured extraction where the instruct model already generates compact outputs, token-level length control adds no value. This is a useful negative result for practitioners.

All code, data, and steering vectors are available at https://github.com/Shumatsurontek/steering-research.

Acknowledgments

Analysis performed using the Transformer Explainer tool [10] and Neuronpedia interpretability platform. Model weights courtesy of the Qwen team [11].

References

[1] A. Turner, L. Thiergart, D. Udell, G. Leech, U. Mini, and M. MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

[2] L. Gao et al. Steering LLM reasoning through bias-only adaptation. In *EMNLP*, 2025. arXiv:2505.18706.

[3] J. Li, W. Zhao, Y. Zhang, and C. Gan. Steering LLM thinking with budget guidance. *arXiv preprint arXiv:2506.13752*, 2025.

[4] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[5] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

[6] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[7] T. Schick et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

[8] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *NeurIPS*, 2024.

[9] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A. Dombrowski, S. Goel, N. Li, M. Lin, S. Wang, C. Xiao, J. Zou, Z. Kolter, and D. Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

[10] H. Cho et al. Transformer Explainer: Interactive learning of text generative models. 2024. https://poloclub.github.io/transformer-explainer/

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.